

CYK algorithm

In computer science, the **Cocke–Younger–Kasami algorithm** (alternatively called **CYK**, or **CKY**) is a parsing algorithm for context-free grammars published by Itiroo Sakai in 1961.^{[1][2]} The algorithm is named after some of its rediscoverers: John Cocke, Daniel Younger, Tadao Kasami, and Jacob T. Schwartz. It employs bottom-up parsing and dynamic programming.

The standard version of CYK operates only on context-free grammars given in Chomsky normal form (CNF). However any context-free grammar may be algorithmically transformed into a CNF grammar expressing the same language (Sipser 1997).

The importance of the CYK algorithm stems from its high efficiency in certain situations. Using big *O* notation, the worst case running time of CYK is $\mathcal{O}(n^3 \cdot |G|)$, where *n* is the length of the parsed string and *|G|* is the size of the CNF grammar *G* (Hopcroft & Ullman 1979, p. 140). This makes it one of the most efficient parsing algorithms in terms of worst-case asymptotic complexity, although other algorithms exist with better average running time in many practical scenarios.

Standard form

The dynamic programming algorithm requires the context-free grammar to be rendered into Chomsky normal form (CNF), because it tests for possibilities to split the current sequence into two smaller sequences. Any context-free grammar that does not generate the empty string can be represented in CNF using only production rules of the forms $A \rightarrow \alpha$ and $A \rightarrow BC$; to allow for the empty string, one can explicitly allow $S \rightarrow \epsilon$, where *S* is the start symbol.^[3]

Algorithm

As pseudocode

The algorithm in pseudocode is as follows:

```
let the input be a string I consisting of n characters:  $a_1 \dots a_n$ .
let the grammar contain r nonterminal symbols  $R_1 \dots R_r$ , with start symbol  $R_1$ .
let  $P[n,n,r]$  be an array of Booleans. Initialize all elements of P to false.
let  $back[n,n,r]$  be an array of lists of backpointing triples. Initialize all elements of back to the empty list.

for each s = 1 to n
  for each unit production  $R_v \rightarrow a_s$ 
    set  $P[1,s,v]$  = true
```

Cocke–Younger–Kasami algorithm (CYK)

Class	Parsing with context-free grammars
Data structure	String
Worst-case performance	$\mathcal{O}(n^3 \cdot G)$, where: <i>n</i> is length of the string <i> G </i> is the size of the CNF grammar

```

for each  $l = 2$  to  $n$  -- Length of span
  for each  $s = 1$  to  $n-l+1$  -- Start of span
    for each  $p = 1$  to  $l-1$  -- Partition of span
      for each production  $R_a \rightarrow R_b R_c$ 
        if  $P[p, s, b]$  and  $P[l-p, s+p, c]$  then
          set  $P[l, s, a] = \text{true}$ ,
          append  $\langle p, b, c \rangle$  to  $\text{back}[l, s, a]$ 

if  $P[n, 1, 1]$  is true then
   $I$  is member of language
  return back -- by retracing the steps through back, one can easily construct all possible parse trees of the
string.
else
  return "not a member of language"

```

As prose

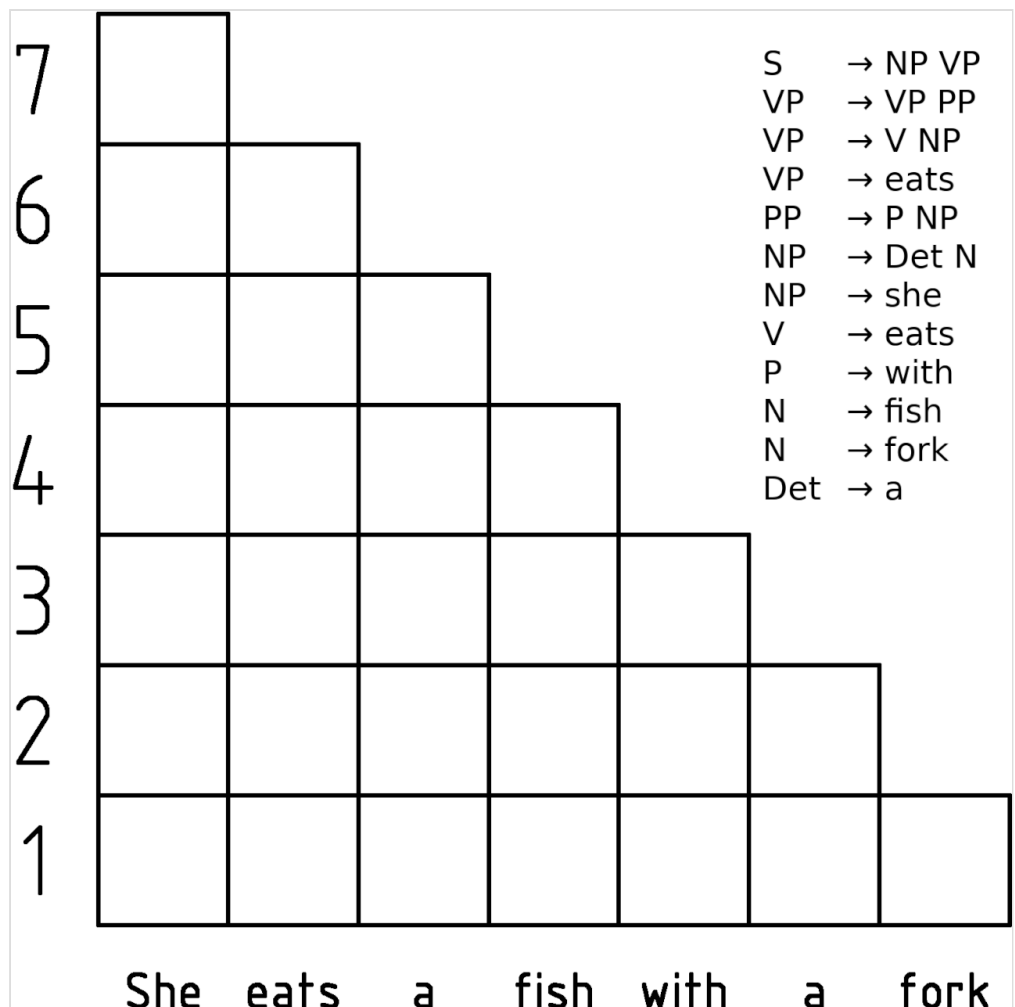
In informal terms, this algorithm considers every possible substring of the input string and sets $P[l, s, v]$ to be true if the substring of length l starting from s can be generated from the nonterminal R_v . Once it has considered substrings of length 1, it goes on to substrings of length 2, and so on. For substrings of length 2 and greater, it considers every possible partition of the substring into two parts, and checks to see if there is some production $A \rightarrow B C$ such that B matches the first part and C matches the second part. If so, it records A as matching the whole substring. Once this process is completed, the input string is generated by the grammar if the substring containing the entire input string is matched by the start symbol.

Example

This is an example grammar:

$S \rightarrow NP VP$
 $VP \rightarrow VP PP$
 $VP \rightarrow V NP$
 $VP \rightarrow \text{eats}$
 $PP \rightarrow P NP$
 $NP \rightarrow \text{Det } N$
 $NP \rightarrow \text{she}$
 $V \rightarrow \text{eats}$
 $P \rightarrow \text{with}$
 $N \rightarrow \text{fish}$
 $N \rightarrow \text{fork}$
 $\text{Det} \rightarrow \text{a}$

Now the sentence *she eats a fish with a fork* is analyzed using the CYK algorithm. In the following table, in $P[i, j, k]$, i is the number of the row (starting at the bottom at 1), and j is the number of the column (starting at the left at 1).



Sentence parsing using the CYK algorithm

CYK table

S						
	VP					
S						
	VP			PP		
S		NP			NP	
NP	V, VP	Det.	N	P	Det	N
she	eats	a	fish	with	a	fork

For readability, the CYK table for P is represented here as a 2-dimensional matrix M containing a set of non-terminal symbols, such that R_k is in $M[i, j]$ if, and only if, $P[i, j, k]$. In the above example, since a start symbol S is in $M[7, 1]$, the sentence can be generated by the grammar.

Extensions

Generating a parse tree

The above algorithm is a recognizer that will only determine if a sentence is in the language. It is simple to extend it into a parser that also constructs a parse tree, by storing parse tree nodes as elements of the array, instead of the Boolean 1. The node is linked to the array elements that were used to produce it, so as to build the tree structure. Only one such node in each array element is needed if only one parse tree is to be produced. However, if all parse trees of an ambiguous sentence are to be kept, it is necessary to store in the array element a list of all the ways the corresponding node can be obtained in the parsing process. This is sometimes done with a second table $B[n,n,r]$ of so-called *backpointers*. The end result is then a shared-forest of possible parse trees, where common trees parts are factored between the various parses. This shared forest can conveniently be read as an ambiguous grammar generating only the sentence parsed, but with the same ambiguity as the original grammar, and the same parse trees up to a very simple renaming of non-terminals, as shown by Lang (1994).

Parsing non-CNF context-free grammars

As pointed out by Lange & Leiß (2009), the drawback of all known transformations into Chomsky normal form is that they can lead to an undesirable bloat in grammar size. The size of a grammar is the sum of the sizes of its production rules, where the size of a rule is one plus the length of its right-hand side. Using g to denote the size of the original grammar, the size blow-up in the worst case may range from g^2 to 2^{2g} , depending on the transformation algorithm used. For the use in teaching, Lange and Leiß propose a slight generalization of the CYK algorithm, "without compromising efficiency of the algorithm, clarity of its presentation, or simplicity of proofs" (Lange & Leiß 2009).

Parsing weighted context-free grammars

It is also possible to extend the CYK algorithm to parse strings using weighted and stochastic context-free grammars. Weights (probabilities) are then stored in the table P instead of Booleans, so $P[i,j,A]$ will contain the minimum weight (maximum probability) that the substring from i to j can be derived from A . Further extensions of the algorithm allow all parses of a string to be enumerated from lowest to highest weight (highest to lowest probability).

Numerical stability

When the probabilistic CYK algorithm is applied to a long string, the splitting probability can become very small due to multiplying many probabilities together. This can be dealt with by summing log-probability instead of multiplying probabilities.

Valiant's algorithm

The worst case running time of CYK is $\Theta(n^3 \cdot |G|)$, where n is the length of the parsed string and $|G|$ is the size of the CNF grammar G . This makes it one of the most efficient algorithms for recognizing general context-free languages in practice. Valiant (1975) gave an extension of the CYK

algorithm. His algorithm computes the same parsing table as the CYK algorithm; yet he showed that algorithms for efficient multiplication of matrices with 0-1-entries can be utilized for performing this computation.

Using the Coppersmith–Winograd algorithm for multiplying these matrices, this gives an asymptotic worst-case running time of $O(n^{2.38} \cdot |G|)$. However, the constant term hidden by the Big O Notation is so large that the Coppersmith–Winograd algorithm is only worthwhile for matrices that are too large to handle on present-day computers (Knuth 1997), and this approach requires subtraction and so is only suitable for recognition. The dependence on efficient matrix multiplication cannot be avoided altogether: Lee (2002) has proved that any parser for context-free grammars working in time $O(n^{3-\epsilon} \cdot |G|)$ can be effectively converted into an algorithm computing the product of $(n \times n)$ -matrices with 0-1-entries in time $O(n^{3-\epsilon/3})$, and this was extended by Abboud et al.^[4] to apply to a constant-size grammar.

See also

- GLR parser
- Earley parser
- Packrat parser
- Inside–outside algorithm

References

1. Grune, Dick (2008). *Parsing techniques: a practical guide* (2nd ed.). New York: Springer. p. 579. ISBN 978-0-387-20248-8.
2. Itiroo Sakai, "Syntax in universal translation". In Proceedings 1961 International Conference on Machine Translation of Languages and Applied Language Analysis, Her Majesty's Stationery Office, London, p. 593-608, 1962.
3. Sipser, Michael (2006). *Introduction to the theory of computation* (2nd ed.). Boston: Thomson Course Technology. Definition 2.8. ISBN 0-534-95097-3. OCLC 58544333 (<https://search.worldcat.org/oclc/58544333>).
4. Abboud, Amir; Backurs, Arturs; Williams, Virginia Vassilevska (2015-11-05). "If the Current Clique Algorithms are Optimal, so is Valiant's Parser". arXiv:1504.01431 (<https://arxiv.org/abs/1504.01431>) [cs.CC (<https://arxiv.org/archive/cs.CC>)].

Sources

- Sakai, Itiroo (1962). *Syntax in universal translation*. 1961 International Conference on Machine Translation of Languages and Applied Language Analysis, Teddington, England. Vol. II. London: Her Majesty's Stationery Office. pp. 593–608.
- Cocke, John; Schwartz, Jacob T. (April 1970). *Programming languages and their compilers: Preliminary notes* (http://www.softwarepreservation.org/projects/FORTRAN/CockeSchwartz_ProgLangCompilers.pdf) (PDF) (Technical report) (2nd revised ed.). CIMS, NYU.
- Hopcroft, John E.; Ullman, Jeffrey D. (1979). *Introduction to Automata Theory, Languages, and Computation* (<https://archive.org/details/introductiontoau00hopc>). Reading/MA: Addison-Wesley. ISBN 0-201-02988-X.
- Kasami, T. (1965). *An efficient recognition and syntax-analysis algorithm for context-free languages* (Technical report). AFCRL. 65-758.

- Knuth, Donald E. (November 14, 1997). *The Art of Computer Programming Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley Professional. p. 501. ISBN 0-201-89684-2.
- Lang, Bernard (1994). "Recognition can be harder than parsing". *Comput. Intell.* **10** (4): 486–494. doi:10.1111/j.1467-8640.1994.tb00011.x (<https://doi.org/10.1111%2Fj.1467-8640.1994.tb00011.x>). S2CID 5873640 (<https://api.semanticscholar.org/CorpusID:5873640>).
- Lange, Martin; Leiß, Hans (2009). "To CNF or not to CNF? An Efficient Yet Presentable Version of the CYK Algorithm" (<http://www.informatica-didactica.de/index.php?page=LangeLeiss2009>). *Informatica Didactica*. **8**.
- Lee, Lillian (2002). "Fast context-free grammar parsing requires fast Boolean matrix multiplication". *J. ACM*. **49** (1): 1–15. arXiv:cs/0112018 (<https://arxiv.org/abs/cs/0112018>). doi:10.1145/505241.505242 (<https://doi.org/10.1145%2F505241.505242>). S2CID 1243491 (<https://api.semanticscholar.org/CorpusID:1243491>).
- Sipser, Michael (1997). *Introduction to the Theory of Computation* (<https://archive.org/details/introductiontoth00sips/page/99>) (1st ed.). IPS. p. 99 (<https://archive.org/details/introductiontoth00sips/page/99>). ISBN 0-534-94728-X.
- Valiant, Leslie G. (1975). "General context-free recognition in less than cubic time" (<https://doi.org/10.1016%2Fs0022-0000%2875%2980046-8>). *J. Comput. Syst. Sci.* **10** (2): 308–314. doi:10.1016/s0022-0000(75)80046-8 (<https://doi.org/10.1016%2Fs0022-0000%2875%2980046-8>).
- Younger, Daniel H. (February 1967). "Recognition and parsing of context-free languages in time n^3 " (<https://doi.org/10.1016%2Fs0019-9958%2867%2980007-x>). *Inform. Control*. **10** (2): 189–208. doi:10.1016/s0019-9958(67)80007-x (<https://doi.org/10.1016%2Fs0019-9958%2867%2980007-x>).

External links

- Interactive Visualization of the CYK algorithm (<https://raw.org/tool/cyk-algorithm/>)
 - CYK parsing demo in JavaScript (<https://martinlaz.github.io/demos/cky.html>)
 - Exorciser is a Java application to generate exercises in the CYK algorithm as well as Finite State Machines, Markov algorithms etc (<https://www.swisseduc.ch/informatik/exorciser/>)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=CYK_algorithm&oldid=1369222226"